Desenvolvimento de Sistemas para Dispositivos Móveis

Camadas, Koin, dados reais, cache e testes de ViewModel

DIM0524 — Turma 01 · Sprint 2 · aula em vídeo

Prof. Fernando · UFRN · 2026.2

Onde paramos

Passo	Hoje
13	Camadas (domínio, dados, apresentação) e injeção de dependência com Koin
14	Dados reais: o app chama a API de Tarefas de Web II com o Ktor Client
15	Abrir sem rede: a última lista guardada no aparelho, com DataStore
16	Testes de ViewModel com Turbine e dublês

Branch `aula`**:** `./passo.sh 12`. Para os passos 14 e 15, a API de Web II precisa estar no ar:

Passo 13

Camadas e injeção de dependência com Koin

Três camadas, em pacotes

Camada	Pode depender de	Não pode depender de
domínio	só Kotlin	Compose, Ktor, DataStore, Koin
dados	domínio, bibliotecas de rede e de disco	apresentação
apresentação	domínio, Compose, ViewModel	detalhes de dados (Ktor, DataStore)

- O ViewModel conhece a interface `RepositorioDeTarefas`, e não quem a implementa
- É o critério "Estado e arquitetura": domínio sem Compose nem infraestrutura, repositórios por interface, sem lógica de negócio dentro de `@Composable`

Injeção de dependência, sem biblioteca

- **Injeção de dependência:** a classe recebe o que usa, em vez de criar. O ViewModel não faz `RepositorioRemoto(...)` por conta própria
- Ganho: o teste passa um dublê pelo mesmo construtor
- Custo: alguém tem de montar a cadeia, e cada objeto deve ser criado uma vez só (um cliente HTTP, um DataStore)
- O Koin é esse "alguém": a montagem vira uma lista de definições, e ele resolve a ordem

Koin: quem monta as peças

- Trocar a implementação (memória → rede) é trocar uma linha no módulo
- Nos testes, o ViewModel é criado com `new`, com o repositório que o teste quiser: sem Koin
- `ModulosTest` confere que o módulo consegue montar o ViewModel

📖 **Ref.** Koin — Compose Multiplatform

O vocabulário do Koin

Peça	O que faz
`module { }`	uma lista de definições: como criar cada coisa
`single { ... }`	cria na primeira vez que alguém pede e reaproveita depois
`factory { ... }`	cria uma instância nova a cada pedido
`get()`	dentro de uma definição: pede ao Koin uma dependência já declarada
`viewModelOf(::TarefasViewModel)`	ViewModel com o ciclo de vida certo; o construtor recebe o que pede
`KoinApplication(application = { modules(...) }) { }`	liga o Koin à interface, em `App()`
`koinViewModel()`	numa função `@Composable`: pede o ViewModel

> Erro comum: esquecer uma definição. O app compila e só quebra ao abrir a tela: "No definition found for type RepositorioDeTarefas".

Conferir o módulo num teste

```
```

- Os outros testes não usam o Koin: criam o ViewModel com o dublê que querem
- Este é o único que liga as peças de verdade, e transforma o erro de execução em erro de teste

Passo 14

Dados reais com o Ktor Client

DTO separado da entidade

Se a API mudar um nome de campo, mudam o DTO e a conversão; o domínio e as telas continuam iguais.

📖 **Ref.** Ktor — Client

O cliente e o repositório remoto

São as rotas `GET` e `POST /tarefas` da API de Tarefas de Web II.

Erros de rede com nome

O que aconteceu	Exceção do Ktor	Falha do domínio	Na tela
API fora do ar, sem rede	`IOException`	`SemConexao`	"Sem conexão com o servidor. Confira a rede e tente de novo."
status 4xx ou 5xx	`ResponseException` (`expectSuccess = true`)	`RespostaInvalida`	"O servidor respondeu algo inesperado (...)"
JSON fora do formato	`ContentConvertException`	`RespostaInvalida`	idem

- Cada falha tem um estado de tela próprio: é o que a rubrica pede em "Dados reais e erro de rede"
- O ViewModel não conhece o Ktor: só as falhas do domínio
- Salvar que falha não apaga a lista: vira um aviso acima dela

Traduzir as exceções da biblioteca

```
```

```
```

- `chamada: suspend () -> T` é uma função como parâmetro: o mesmo tratamento serve para `listar` e `adicionar`
- O `catch` pega só as falhas do domínio: o cancelamento segue adiante sozinho

Onde está a API

Onde o app roda	Endereço da API
desktop, na mesma máquina	`http://localhost:8080`
emulador Android	`http://10.0.2.2:8080`: o computador que roda o emulador
sem a API (Codespace, laboratório)	`TAREFAS_API=memoria` ou `mise run app:sem-api`: repositório em memória
celular na mesma rede Wi-Fi	o IP do computador, por exemplo `http://192.168.0.10:8080`

No Android: permissão `INTERNET` e, para `http://` local, `usesCleartextTraffic="true"` (num app publicado, a API usa `https` e essa linha sai).

Demonstração: a API de Web II no ar, `./passo.sh 14`, `mise run app`: as tarefas vêm do banco. Pare a API e toque em **Tentar de novo**.

Testar o repositório sem a API

```
```

- O `MockEngine` responde o que o teste mandar: JSON certo, status 500, JSON fora do formato, conexão recusada
- São os 4 testes de `RepositorioRemotoTest`, e rodam sem a API de Web II

Passo 15

Abrir sem rede

Um repositório que usa outro

- O ViewModel não sabe que existe cache: só recebe `doCache = true`, e a tela mostra o aviso
- Para listas grandes ou consultas, um banco no aparelho (Room ou SQLDelight); para uma lista pequena, o DataStore basta

📖 **Ref.** DataStore — Kotlin Multiplatform

O cache com DataStore

- O DataStore guarda pares chave e valor num arquivo, de forma assíncrona: `edit` e `data` não travam a tela
- `data` é um `Flow`: `first()` lê o valor atual uma vez
- A lista vai como o JSON dos DTOs, o mesmo formato que vem da API

Um repositório que envolve outro

Situação	O que `listar()` devolve
a API respondeu	a lista nova, que também é guardada
sem conexão, com algo guardado	a última lista, com `doCache = true`
sem conexão, nada guardado	a falha `SemConexao` continua: a tela mostra o erro
resposta inválida	a falha continua: o cache só cobre a falta de conexão

- Um objeto que implementa a mesma interface que recebe e acrescenta um comportamento: o padrão **Decorator**
- O ViewModel não muda: o Koin entrega o repositório com cache no lugar do remoto

Onde fica o arquivo

Plataforma	App(diretorioDeDados = ...)
Android (`MainActivity`)	`filesDir.path`, a pasta privada do app
desktop (`main.kt`)	`~/.tarefas-compose`

Demonstração: abrir o app com a API no ar; fechar; parar a API (`docker compose stop api-ktor`); abrir de novo: "Sem conexão: mostrando a última lista salva."

Onde guardar dados no aparelho

Opção	Para quê	No projeto
DataStore (Preferences)	pares chave e valor; configurações; um cache pequeno	o exemplo guarda a última lista numa chave só
Room	banco SQLite com consultas conferidas na compilação; Kotlin Multiplatform	listas grandes, buscas, relações entre dados
SQLDelight	você escreve o SQL, e ele gera o código Kotlin; Kotlin Multiplatform	idem, para quem prefere escrever o SQL

Dois cuidados com o DataStore:

- **Um DataStore por arquivo, por processo.** Dois para o mesmo arquivo é erro; o Koin cria um só (`single`)
- **Cada plataforma tem a sua pasta**: o diretório chega de fora, como parâmetro de `App`

A rubrica pede que o app abra sem rede mostrando o último conteúdo; a forma de guardar fica a critério do grupo.

Passo 16

Testes de ViewModel com Turbine

Um dublê e o relógio virtual

```
```

```
```

- `awaitItem()` é o próximo estado emitido, na ordem: o teste vê a sequência, e não só o fim
- O tempo é virtual: os 500 ms do dublê passam na hora

📖 **Ref.** Turbine · Android — Testing coroutines

Turbine: a sequência inteira

```
```

- O dublê muda de comportamento no meio do teste: primeiro falha, depois responde
- `skipItems(1)` pula o que não interessa; `awaitItem()` falha se o estado esperado não chegar
- No fim do bloco, o Turbine confere que não sobrou estado sem verificação

Dois despachantes de teste

	`StandardTestDispatcher(testScheduler)`	`UnconfinedTestDispatcher()`
quando a corrotina roda	entra numa fila e roda quando o teste espera	na hora, até a primeira suspensão
no exemplo	testes de ViewModel com Turbine: é possível ver o `Carregando`	testes de tela e o `ModulosTest`: só o resultado importa

Esquecer o `resetMain` faz o próximo teste herdar o despachante do anterior: falhas que dependem da ordem dos testes.

A suíte, no fim do Passo 16

Arquivo	Testes	O que cobre
`TarefasViewModelTest`	7	carregando → sucesso, vazio, sem conexão, resposta inválida, tentar de novo, lista do cache, adicionar
`TelasTest`	7	formulário, linha alternável, cabeçalho, navegação, janela larga, erro com tentar de novo, aviso de cache
`RepositorioRemotoTest`	4	JSON → tarefas, servidor fora, 500, JSON fora do formato (`MockEngine`)
`RepositorioComCacheTest`	3	guarda, devolve sem conexão, erro sem nada guardado
`RegrasTest` e `ModulosTest`	4	regras puras e o módulo do Koin

A rubrica pede ≥ 5 de interface e ≥ 5 de ViewModel, verdes no CI.

Para conferir o entendimento

1. Por que o ViewModel recebe `RepositorioDeTarefas`, e não `RepositorioRemoto`?
2. Qual a diferença entre `single` e `factory` no Koin, e por que o DataStore é `single`?
3. O que acontece com uma resposta 500 se o cliente não tiver `expectSuccess = true`?
4. Por que a falta de conexão usa o cache e a resposta inválida não?
5. O que o Turbine mostra que um `assertEquals` no estado final não mostra?

O que a Sprint 2 cobra

Critério	Peso	O que mostrar
Estado e arquitetura	25%	ViewModel com `StateFlow`; camadas em `commonMain`; domínio sem Compose nem infraestrutura; repositórios por interface
Estados da interface	15%	carregando, erro, vazio e sucesso, por classe selada, na tela
Dados reais e erro de rede	25%	API real, Ktor Client, DTO separado; falta de conexão e resposta inválida com estado próprio
Persistência local	10%	o app abre sem rede mostrando o último conteúdo
Testes	15%	≥ 5 de interface e ≥ 5 de ViewModel com Turbine e dublês, verdes no CI
App e documentação	10%	sem crash; `docs/arquitetura.md`; README; APK numa release

Prova em **11/11**; *daily meetings* em 23 e 25/11; **entrega em 30/11**.

Onde estudar depois

- Leitura da sprint: `leituras/moveis-s2.md`, capítulos 5 a 9
- Passos: `exemplos/tarefas-compose/PASSOS.md` · branch `aula`, tags `passo-13` a `passo-16`
- Koin — Compose Multiplatform
- Ktor — Client
- DataStore — Kotlin Multiplatform
- Turbine · Android — Testing coroutines